VEGA EDULINK

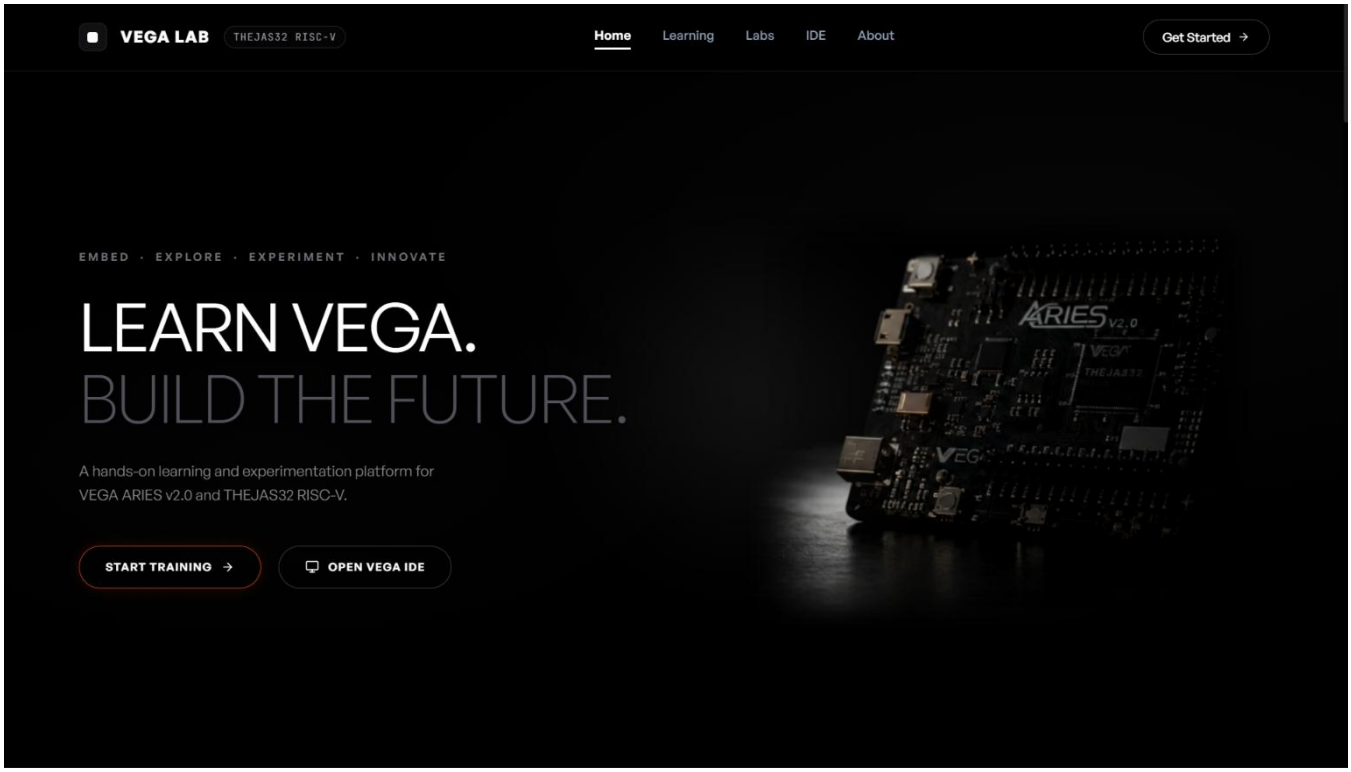
An Integrated Web IDE, OTA Programming System and Trainer Kit for Beginner-Friendly VEGA Microcontroller Learning and Development

Learn • Code • Build • Flash • Experiment

Project Website: https://vega-web-ide.vercel.app/

Project Team

Team Member	Contribution
Haresh S	Embedded Software & Web IDE Developer
Chandru Krishnakumar	Embedded Software & Hardware Integration | PCB Design
Rishikesh S	Embedded Hardware, Firmware & OTA Integration
Dhayananth A	Hardware, Trainer Kit & Project Testing



1. Introduction

Learning a new microcontroller can be difficult for beginners when learning materials, programming tools, libraries, examples, documentation and hardware experiments are available in different places. VEGA EDULINK was developed to bring these activities together in a single, beginner-friendly environment for learning and developing with the VEGA ARIES v2.

The project combines the VEGA WEB IDE with a hands-on VEGA Trainer Kit. Together, they create a continuous path from learning basic concepts to writing programs, generating firmware, programming the board and testing the result on real hardware.

2. Project Objective

The main objective of VEGA EDULINK is to provide an all-in-one platform where students can learn the fundamentals required for VEGA programming, write and build programs, manage libraries, access examples and technical resources, and program the VEGA ARIES v2 through a hands-on Trainer Kit.

- Provide structured learning content for beginners.
- Provide a browser-based coding environment for VEGA programs.
- Allow libraries, examples and technical resources to be accessed from one platform.
- Generate the required .bin firmware using the VEGA RISC-V toolchain.
- Support Direct USB Flashing and OTA Flashing.
- Provide practical GPIO and communication-protocol experiments through the Trainer Kit.

3. Student Learning Journey

The student first opens the VEGA EDULINK website and selects Start Learning. The learning section introduces the basic concepts needed for programming the VEGA microcontroller. After learning the fundamentals, the student opens the VEGA WEB IDE, writes a program, selects or installs the required libraries and builds the firmware.

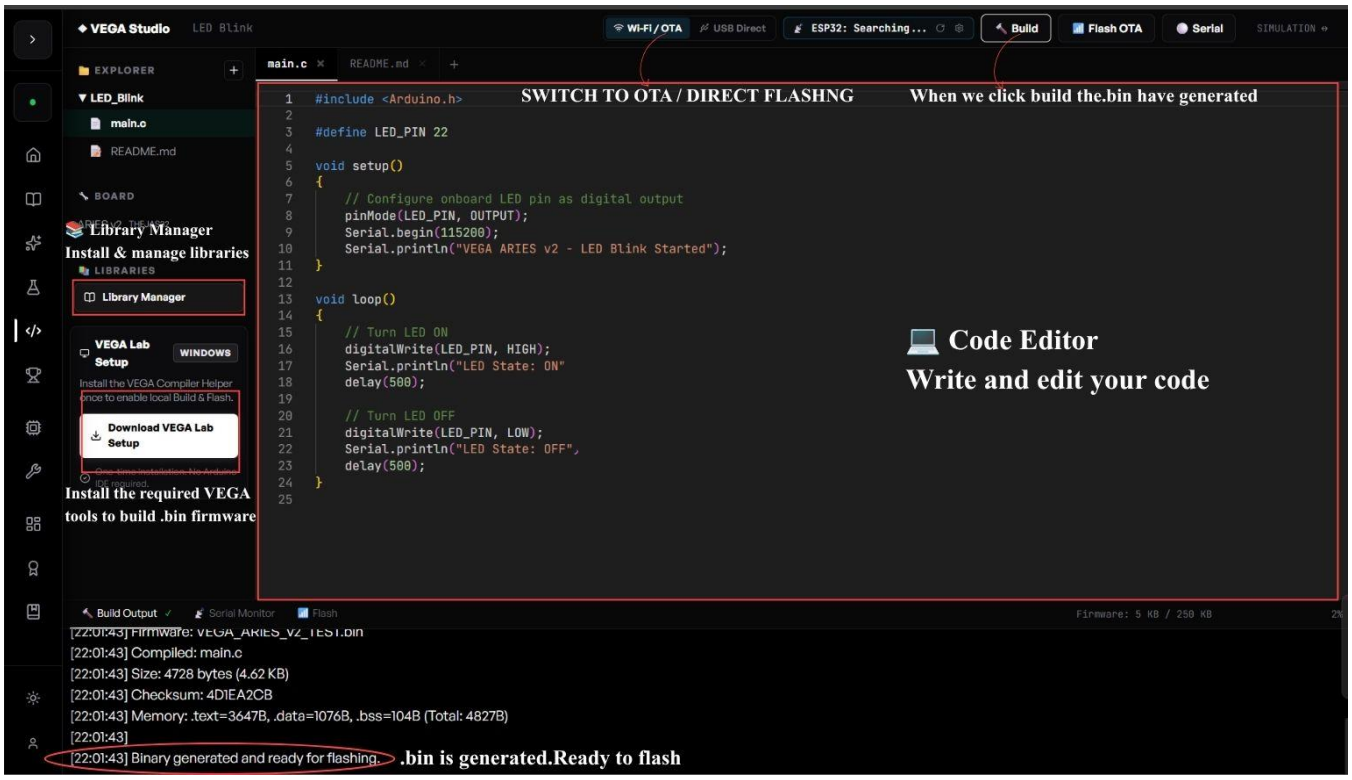
After the firmware is generated, it can be flashed to the VEGA ARIES v2 through Direct USB or OTA. The programmed board is then used with the Trainer Kit so the student can observe the result on real peripherals.

Complete workflow

Learn → Code → Build → Flash → Experiment → Build Projects

4. VEGA WEB IDE

The VEGA WEB IDE is the software development environment of **VEGA EDULINK**. It allows students to write and edit VEGA programs through a web interface while providing examples, library management and supporting learning resources.



VEGA EDULINK website and web-based learning environment

- Start Learning – access concepts and learning material.
- Web IDE – write and edit VEGA programs.
- Examples – use example programs for experiments.
- Library Manager – install and manage required libraries.
- Resources – access documentation, pinouts, schematics and related material.
- Build and Flash – generate firmware and program the VEGA ARIES v2.

5. Firmware Build Process

5.1 VEGA Package Installation

The Web IDE provides the required **VEGA package** through the **VEGA Lab Setup**. The student downloads and installs the package on the computer before using the Build feature.

The package prepares the local development environment with the required components for VEGA firmware generation, including:

- **VEGA RISC-V cross-compiler**
- VEGA Arduino core
- System and processor headers
- Board variant files
- Required libraries
- Linker script (link1.lds)
- Required build tools

Once the package is installed, the Web IDE's local build service can access these components and use them during compilation.

5.2 Source Code Compilation

After installing the VEGA package, the student opens the **VEGA WEB IDE**, writes the required C/C++ program in the code editor, and clicks the **BUILD** button. The Web IDE sends the source code to the local VEGA build service, which manages the complete compilation process.

The build service prepares the required build environment by configuring the **VEGA compiler options, target architecture, header files, core files, and library paths**. It also identifies the libraries required by the program and makes their corresponding headers and source files available during compilation.

The project uses the **VEGA RISC-V cross-compiler**, such as:

riscv32-vega-elf-g++

The cross-compiler is required because the source code is being compiled on the student's computer, while the generated machine instructions need to execute on the **THEJAS32 RV32IM processor** used by the VEGA ARIES v2.

The compiler is configured for the VEGA target using the appropriate architecture and ABI settings, including:
-march=rv32im
-mabi=ilp32

During compilation, the student's source files and the required library source files are processed by the compiler. The compiler translates the C/C++ code into machine code and produces **object (.o) files**.

For example, if a program uses a library through:
#include <SPI.h>

the corresponding VEGA library is included in the build environment, and its required source files are compiled along with the student's program.

This allows the Web IDE to automatically handle the required compilation dependencies instead of requiring the student to manually configure them.

C/C++ Source Code
↓
Local VEGA Build Service
↓
Configure Compiler & Build Settings
↓
VEGA RISC-V Cross-Compiler
↓
THEJAS32 RV32IM Target
↓
Required Libraries
↓
Object Files (.o)

5.3 Linking and Firmware Binary Generation

After the compilation stage, the generated object files are combined with the required **VEGA Arduino core, libraries, runtime components, and linker script (link1.lds)**. The linker resolves the dependencies between these components and creates the complete **ELF (.elf) firmware image** for the THEJAS32 processor. The link1.lds linker script defines the required memory layout and placement of the different firmware sections.

The generated ELF file is then processed using the **objcopy** tool to convert the required firmware contents into the binary format required for programming the VEGA ARIES v2. This produces the final **firmware.bin** file.

The build service then verifies that the binary has been generated successfully. If the complete build process is successful, the Web IDE displays **BUILD SUCCESS** and retains the generated firmware.bin for the subsequent **Direct USB or OTA flashing process**.

6. Programming Methods

6.1 Direct USB Flashing

In Direct USB mode, the laptop is connected directly to the VEGA ARIES v2. The generated .bin firmware is transferred through the USB programming path.

Direct USB programming supports both XMODEM-based bootloader programming and flasher-based programming.

Flow:

Web IDE → Generate .bin → USB → VEGA ARIES v2 → XMODEM / Flasher → Flash Memory

6.2 OTA Flashing

The VEGA EDULINK platform enables wireless firmware programming of the VEGA ARIES V2 RISC-V microcontroller using an **ESP32-S3 Wi-Fi module as a programming gateway**.

1. Firmware Generation and Upload

The user writes C/C++ application code in the **VEGA Web IDE** and initiates the build process. The backend generates the application **.bin firmware image**, which is sent from the website to the ESP32-S3 through Wi-Fi.

The ESP32-S3 runs an **HTTP server** to receive the firmware and uses a **Wi-Fi Manager** for network configuration. The received application .bin is stored in **LittleFS**, and its **SHA-256 hash (for firmware integrity verification)** is calculated.

2. USB Host and VEGA Connection

The ESP32-S3 starts its **USB Host interface** and detects the **FT230X USB-to-UART bridge**, establishing serial communication with the VEGA ARIES V2.

The USB interface also provides connection monitoring and recovery handling when required.

3. VEGA Bootloader Handshake

After detecting the FT230X, the VEGA ARIES V2 is **manually reset** to enter the ROM bootloader.

The synchronization sequence is:

< → > → C

- **<** → VEGA bootloader requests synchronization
- **>** → ESP32-S3 acknowledges the bootloader
- **C** → VEGA indicates readiness for XMODEM-CRC transfer

4. VEGA Flasher Transfer

The **VEGA flasher binary is embedded inside the ESP32-S3 firmware as an internal binary array**.

After the VEGA enters the ROM bootloader, the ESP32-S3 **first transfers this embedded VEGA flasher binary to the VEGA** using XMODEM-CRC. This flasher is transferred before the application firmware and is intended to execute from VEGA RAM.

XMODEM-CRC Packet Format

SOH | Block Number | Complement | 128-Byte Data | 16-bit CRC

Transfer Control

Control	Function
ACK	Block accepted
NAK	Retransmission requested
CAN	Transfer cancelled
EOT	End of transmission
Timeout + Retry	Handles communication failures

5. Flasher Execution and Flash Preparation

After the flasher binary transfer is completed, the **VEGA flasher executes from VEGA RAM**.

The flasher then prepares the **external SPI Flash**, including the required erase operation.

The ESP32-S3 does **not directly erase or program the external SPI Flash**. It only provides the firmware data to the VEGA through the serial transfer. The **VEGA-resident flasher performs the actual Flash erase and programming operations**.

6. Application Firmware Transfer

Once the external SPI Flash is prepared, the VEGA flasher sends **C**, indicating that it is ready to receive the application image.

The ESP32-S3 then reads the **application .bin previously stored in LittleFS** and transfers it to the VEGA flasher using the same XMODEM-CRC mechanism.

Application Transfer

128-Byte Data Block → 16-bit CRC → ACK / NAK → Retry

The VEGA flasher receives the application firmware blocks and writes them to the **external SPI Flash**.

7. Programming Status

The programming process is managed using the following states:

IDLE → DEVICE_CONNECTED → WAITING_FOR_BOOTLOADER → PROGRAMMING → SUCCESS

If a programming operation fails, the system can enter **FAILED_RETRY** for recovery.

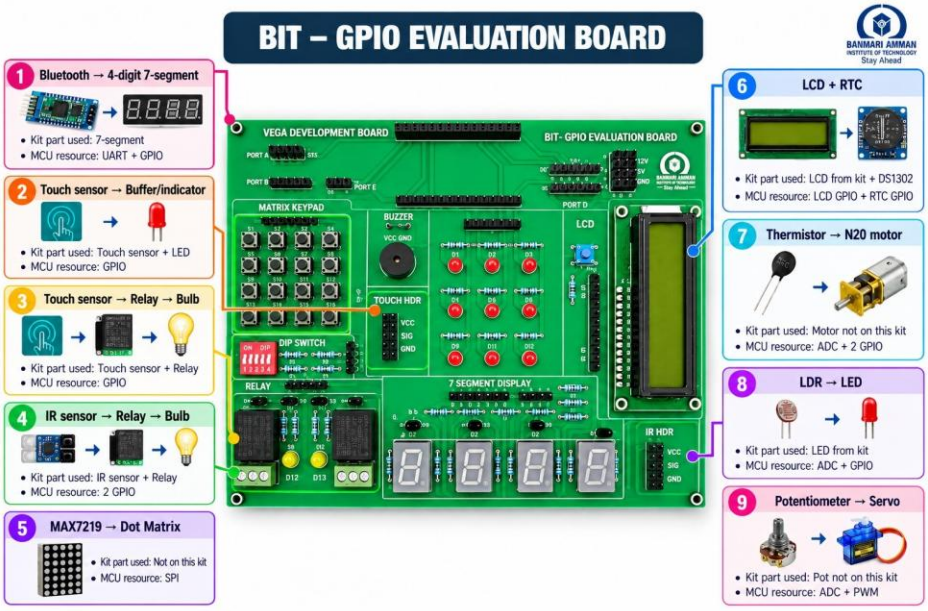
VEGA Web IDE
↓
Generate Application .bin
↓
Wi-Fi → ESP32-S3
↓
HTTP Server → LittleFS
↓
USB Host → FT230X → UART
↓
VEGA Reset → ROM Bootloader
↓
< → > → C (Bootloader Handshake)
↓
Embedded VEGA Flasher Binary
↓
XMODEM-CRC → VEGA
↓
Flasher Executes from VEGA RAM
↓
Prepare / Erase External SPI Flash
↓
C (Ready for Application Firmware)
↓
Application .bin Read from LittleFS
↓
XMODEM-CRC → VEGA Flasher
↓
VEGA Flasher → External SPI Flash
↓
SUCCESS

7. VEGA Trainer Kit

The VEGA Trainer Kit provides the hands-on hardware environment for applying the concepts learned through VEGA EDULINK.

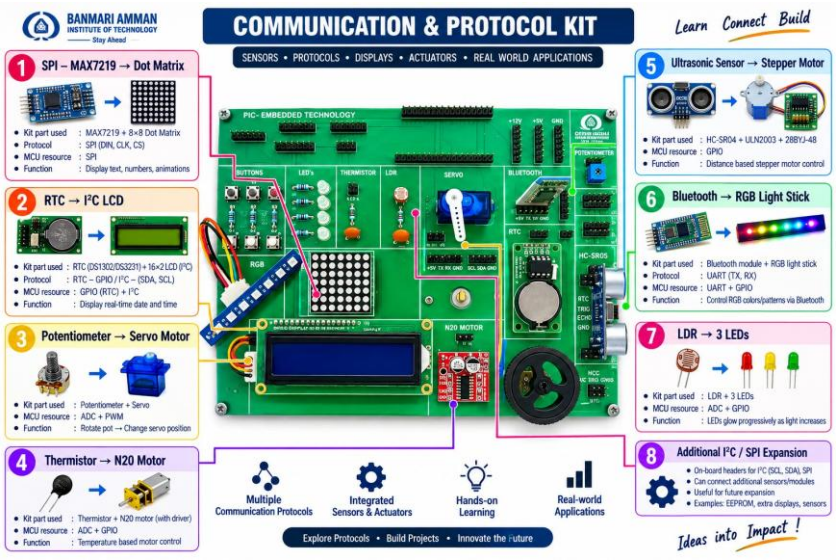
7.1 GPIO Trainer Board

The GPIO Trainer Board provides GPIO-based peripherals and interfaces. The GPIO pins of the VEGA ARIES v2 are connected to the corresponding GPIO pins on the trainer board for digital input and output experiments.



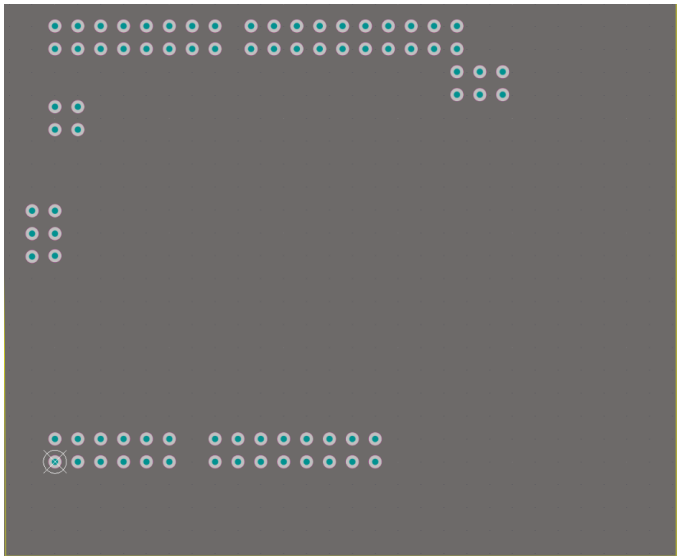
7.2 Communication Protocols Trainer Board

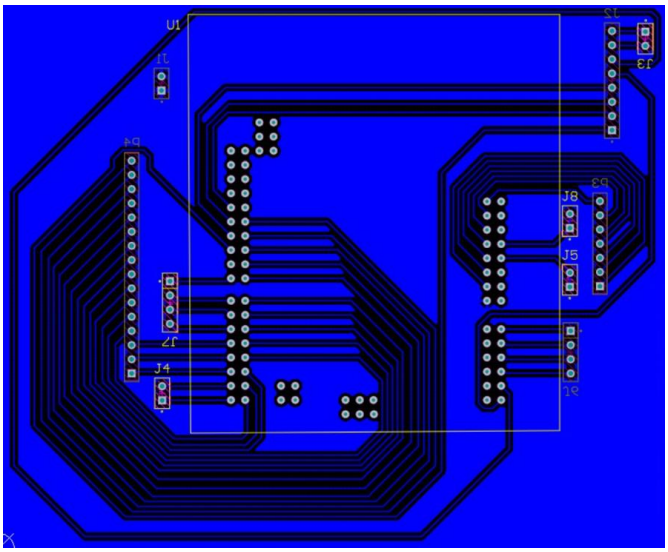
The Communication Protocols Trainer Board is designed for UART, SPI and I^2C experiments. The corresponding communication pins of the VEGA ARIES v2 are connected to the respective interfaces on the trainer board.



7.3 VEGA Interconnection Board and Footprint

A dedicated VEGA Interconnection Board was developed to provide the required connection and signal routing between the VEGA ARIES v2 and the two trainer boards. A dedicated footprint for the VEGA ARIES v2 was also created to support proper placement, alignment and integration of the board.





8. Coding and Libraries

Trainer Kit programs are developed using the Arduino-compatible programming environment for the VEGA ARIES v2. GPIO experiments configure the required pins as inputs or outputs, while communication experiments initialize UART, SPI or I^2C according to the connected peripheral.

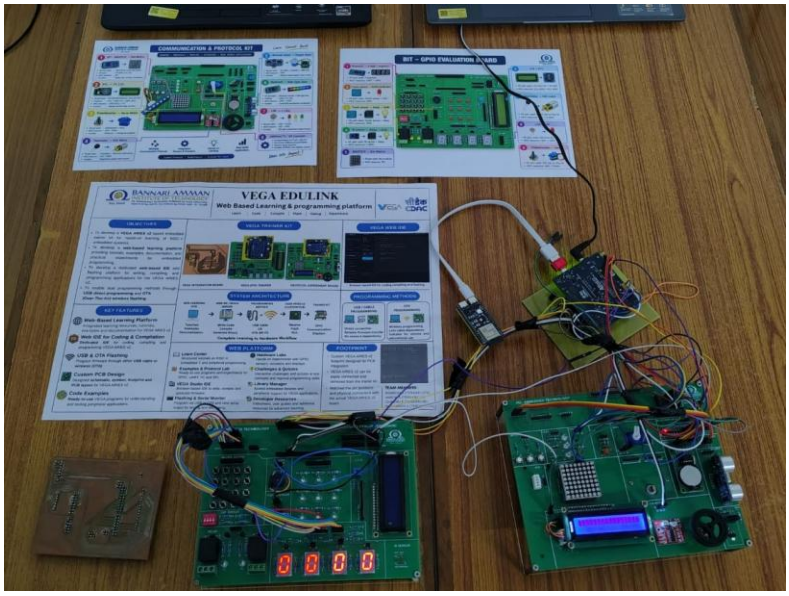
The VEGA WEB IDE Library Manager is used to install and manage libraries required for different experiments, allowing students to use the appropriate libraries directly from the development platform.

9. Testing and Demonstration

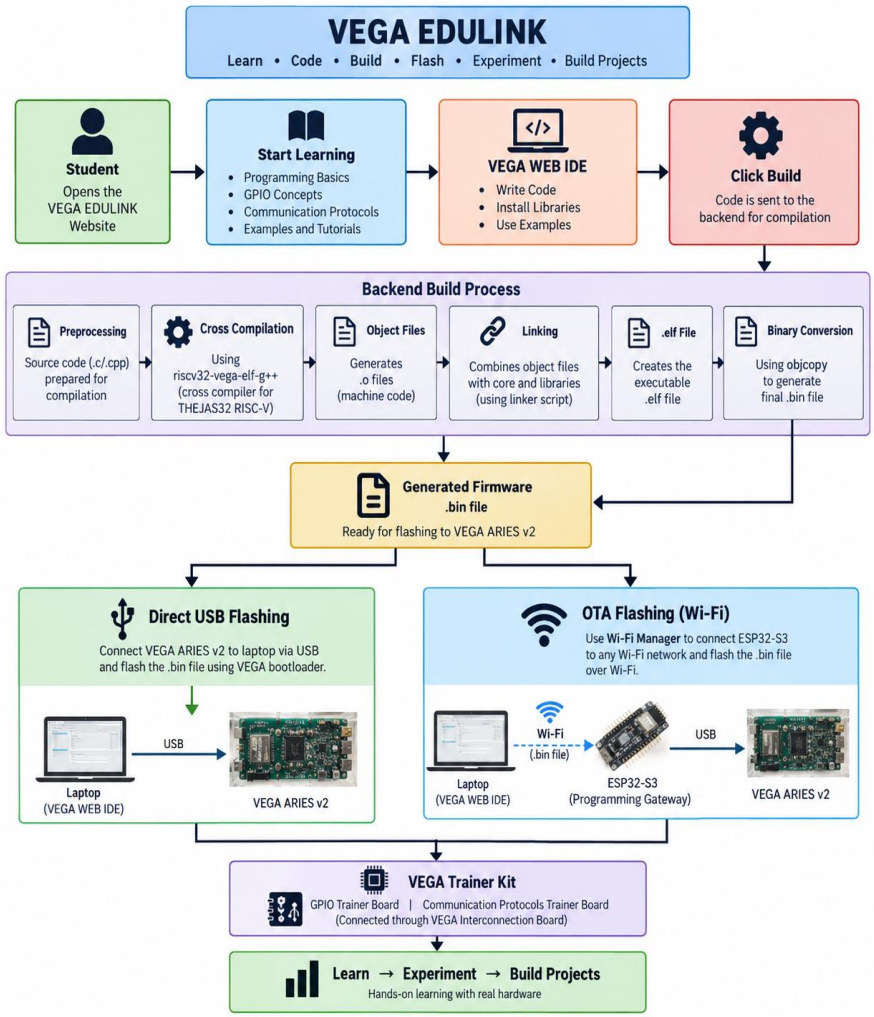
The complete workflow was tested from firmware generation to real hardware operation. The required connections between the VEGA ARIES v2, VEGA Interconnection Board, and trainer boards were verified before testing.

The firmware was successfully generated through the VEGA WEB IDE and flashed to the VEGA ARIES v2 through OTA using the ESP32-S3 gateway. Bluetooth communication was then tested by sending values from a mobile phone to the VEGA ARIES v2 through the Bluetooth module. The received values were processed by the VEGA board and displayed on the 7-segment display, confirming the communication and display operation.

Demonstrated Hardware Flow
Mobile Phone → Bluetooth → VEGA ARIES v2 → 7-Segment Display



10. Overall System



11. Project Outcome

VEGA EDULINK connects learning, coding, firmware generation, programming and hands-on experimentation into one workflow. The combination of the Web IDE and Trainer Kit helps beginners move from basic concepts to practical VEGA-based applications.

- Beginner-focused learning and development workflow.
- Integrated web-based coding environment.
- VEGA-specific .bin firmware generation.
- Direct USB and Wi-Fi-based OTA programming.
- Hands-on GPIO and UART/SPI/I^2C experimentation.
- Dedicated interconnection and VEGA board footprint for Trainer Kit integration.

12. Achievement and Links

🥉 3rd Prize Winner – Hackathon

Video Link: VEGA WEB IDE OUTPUT VIDEO

GitHub Link: VEGA WEB IDE

13. Conclusion

VEGA EDULINK was developed with a simple objective: make learning and programming the VEGA microcontroller more accessible to beginners. By combining structured learning, a web-based IDE, library and resource access, firmware generation, Direct USB and OTA programming, and a practical Trainer Kit, the project connects software learning with real hardware experimentation.

The workflow enables students to understand a concept, write code, generate firmware, program the VEGA ARIES v2 and observe the result on the Trainer Kit. This provides a practical path from learning to experimentation and project development.